

OpenCam - Milestone 3 Report

Security findings, implemented changes, verified behavior, unresolved V4L2/FFmpeg failure, and a manual troubleshooting plan

Report date: 28 September 2026

Repository: OpenCam Linux, Android, and legacy Windows implementations

Evidence base: M3_SECURITY_AUDIT.md, current source tree, recorded Linux/Android integration session, and build/unit-test results documented in that audit.

Purpose: provide an evidence-based handoff and practical manual next steps. This report does not claim the end-to-end virtual-camera pipeline is complete.

1. Executive summary

The remediation substantially improved OpenCam's security boundaries and fixed the original Wi-Fi connection failures. The physical Android phone completed the new one-time QR pairing, mutual control authentication, descriptor exchange, activation, camera startup, and authenticated RTSP readiness while the USB cable remained connected and Wi-Fi was explicitly selected.

The remaining blocker is the actual virtual-camera output. Linux started FFmpeg, then observed child exit status 8. The process redirected FFmpeg stderr to /dev/null, so the reason is unknown. A post-exit query of /dev/video2 returned invalid format/parameter errors; no active FFmpeg writer or delivered frames were observed. Therefore the complete Android camera -> RTSP -> FFmpeg -> V4L2 pipeline is not yet verified as working.

Do not weaken QR authentication, session authentication, URL allowlisting, or the RTSP credential merely to get past this output failure. The known connection bugs were fixed without an authentication bypass. The next best step is to retain protected FFmpeg error diagnostics, verify the OpenCam loopback device and accepted output caps, then reproduce a short local test-pattern write before attempting another camera session.

Overall assessment: security materially improved; Wi-Fi control integration verified; media-input readiness verified; V4L2 output unresolved; not suitable to describe as fully secure or fully operational on hostile networks.

2. Scope, provenance, and confidence

This is a synthesis of the audit and remediation/integration evidence already recorded in the repository. It is not a new penetration test, and no new network or device actions were performed to create this PDF. Historical findings in Sections 1-23 of the audit describe the pre-remediation system; Sections 24-26 document later source changes and physical-device observations.

Evidence label | Meaning in this report

Confirmed by source | The checked-in source path directly demonstrates the behavior or the mitigation.

Confirmed at runtime | The behavior was observed in the recorded local integration run.

Partial / unresolved | Some controls or stages are verified, but a remaining implementation limit or missing runtime test prevents a complete claim.

Unknown | The evidence does not determine the answer; do not convert this to a confirmed vulnerability or confirmed fix.

Source paths are cited by repository-relative file and function/component. The full evidence, original finding text, and historic runtime snapshots remain in M3_SECURITY_AUDIT.md.

3. System architecture and trust boundaries

Android CameraX camera | -> | Android RTSP server :8554 / random session path | -> | Linux fprobe, then FFmpeg
| -> | OpenCam V4L2 loopback /dev/video*

^ authenticated activation/control over Wi-Fi TCP/6969; Android initiates using OCAM1 QR

Linux controller displays one-time endpoint + pairing secret in QR; peer authentication and session integrity are checked before descriptor acceptance.

Control plane: Android initiates TCP to Linux port 6969 using the endpoint and 256-bit pairing secret encoded in the new QR format. Linux and Android perform mutual HMAC-based authentication, derive session state, exchange framed messages, send a device descriptor, and activate the camera. The protocol is authenticated/integrity-protected, but not encrypted; an on-path observer can still see control metadata and potentially capture a

pairing secret during its validity window.

Media plane: Android hosts RTSP; Linux is the ffprobe/FFmpeg client. The path is random per stream and Wi-Fi sessions use a session-derived Basic credential. These are separate protections from control authentication. The Android RTSP library's auth coverage is incomplete, it binds wildcard IPv4, and RTSP has no TLS in the current setup. Treat media access as partially protected, not confidential or fully authorized.

USB/ADB: the Android UI now has explicit persisted Wi-Fi/USB selection; cable attachment does not silently switch modes. The tested successful route was Wi-Fi while USB remained attached. The Linux controller does not currently provide the complete ADB reverse/forward path needed for end-to-end USB transport, and the legacy Windows ADB path is distinct. The USB selection must not be represented as verified for Linux until both the control and RTSP forwarding directions are implemented and tested with the authenticated protocol.

4. Network attack surface and trust assessment

Endpoint / path | Current role and controls | Residual concern

Linux TCP/6969 | Android initiates IPv4 control. Linux route-selects an IPv4 address and binds the listener to that address. QR carries OCAM1, endpoint and 256-bit random secret; secret expires after ten minutes and rotates after successful pairing. Versioned frames, 64 KiB max, timeouts, mutual HMAC proofs and sequenced session HMACs are implemented. | No TLS/confidentiality. Route selection may choose a VPN/default route. IPv6 intentionally unsupported. Single serialized handler remains. Firewall policy is outside application control.

Android RTSP TCP/8554 | Android hosts the camera media server at a random per-session path. Wi-Fi credential is derived from the authenticated session; Linux accepts only the authenticated peer address, port 8554 and expected path. ffprobe/FFmpeg use RTSP over TCP allowlists. | Library binds wildcard IPv4; its Basic auth does not protect every method; no TLS. Other LAN clients may reach metadata/media depending on method and library behavior. Independent retrieval and interface reachability are not established.

Linux ffprobe / FFmpeg egress | Linux clients probe and consume the Android RTSP endpoint. URL is checked before process launch. Child receives argv directly, not a shell string; unrelated inherited file descriptors are closed. | Authentication does not encrypt media. Diagnostics must never leak URL userinfo/session password. Runtime FD-inheritance regression still needs checking after the fix.

USB/ADB path | Windows legacy implementation configures ADB reverse/forward; its server is restricted to loopback. Android now exposes a user-selected USB mode. | There is no complete Linux ADB path integrated with the current authenticated Linux listener. UI selection is not proof of transport support.

Same-LAN attacker: before remediation, unauthenticated TCP/6969, unbounded descriptor handling and the shell sink made the host highly exposed if reachable. After remediation, a peer must have the short-lived pairing secret and valid session proof to reach descriptor/media setup. A LAN observer can still see plaintext control/media traffic; the RTSP library's incomplete authorization remains material.

Rogue Android/Linux endpoint: mutual HMAC authentication now rejects a wrong pairing secret and proves the QR secret holder; Android and Linux share authenticated session frames. A one-time QR is still a bearer secret, so protect the display and avoid screenshots/logs. This is authentication, not server certificate identity backed by a CA/public-key pin, nor encryption.

Malformed client / resource abuse: strict framed lengths, field caps, parser validation and deadlines address the historic unbounded-memory and parser risks. Finite serial handling still leaves bounded queue delay. Oversized/malformed runtime cases after remediation need regression/security tests.

Multi-interface: Linux route-aware IPv4 selection and specific-address binding improve on first-interface/wildcard behavior. It has no IPv6 listener by design; VPN/virtual-interface choices and firewall reachability need testing in the actual deployment.

5. Why the connection initially failed-and what fixed it

- Automatic USB preference plus missing Linux ADB path. Earlier Android startup chose localhost/ADB just because a USB cable existed; Linux had no matching ADB reverse mapping. A separate attempt had Linux and the phone on different IP subnets. A visible, persisted mode selector now makes Wi-Fi explicit, and the device successfully used Wi-Fi with USB still plugged in.

- Authenticated full-duplex lock deadlock. The control channel's blocking receive and descriptor send shared one synchronization lock. The reader waited for activation while the sender needed the same lock to send the descriptor. Separate send and receive locks fixed the deadlock. Android test `blockedReceiveDoesNotBlockFullDuplexDescriptorSend` covers the regression.

- Stale media-password length guard. Session-derived RTSP credentials were standardized as 64 lowercase hexadecimal characters, but a Linux guard still expected a prior 43-character Base64 representation. Updating the guard allowed the authenticated activation/RTSP path to proceed. Cross-platform deterministic vectors verify the derivation.

- RTSP probe deadline too short. The previous four-second ffprobe parent limit could expire before FFprobe's

normal stream analysis completed. The probe now uses a one-second analysis duration and an eight-second hard deadline. On the final integration run Linux reported "RTSP server is ready".

These changes address actual integration causes; none disables pairing authentication or accepts endpoint-only legacy QR data.

6. Security remediation summary: M3-01 through M3-11

ID / pre-remediation issue | Implemented change and current status | Remaining evidence gap / limitation

M3-01 · HIGH Peer URL interpolated into shell-mediated fprobe command. Evidence: linux/src/rtsp_probe.cpp::waitForRtsp/runProbe. | FIXED in source/tests. Probe uses fork/exec argv, bounded wait/termination/reaping. Harmless shell-marker regression passed. | No exploit payload was run; that is intentional. The static sink is gone and shell regression is the appropriate evidence.

M3-02 · MEDIUM Unauthenticated peer controls descriptor, URL and FFmpeg protocol. Evidence: linux/src/main.cpp::handleClient; security_validation.cpp; ffmpeg_runner.cpp; Windows net/server.cpp. | PARTIALLY FIXED. Short-lived single-use QR secret, mutual HMAC, sequenced authenticated session frames, strict peer/port/path checks, RTSP/TCP protocol whitelist. | Control and RTSP are plaintext. Captured QR secret may be used until pairing/expiry. Stronger network confidentiality and adversarial session tests remain.

M3-03 · MEDIUM Unbounded descriptor accumulation and synchronous starvation. Evidence: linux/src/main.cpp::handleClient; device_descriptor.cpp; current_control_protocol.cpp. | PARTIALLY FIXED. Framed 64 KiB cap, bounded fields/counts, 5 s handshake and 30 s descriptor deadlines, strict parser. | Server remains serial; an authenticated slow client can delay subsequent users within the timeout. No stress/load test was run after fixes.

M3-04 · MEDIUM Android treated arbitrary TCP chunks as complete messages and mishandled EOF. Evidence: Android TCPConnection.kt, ControlProtocol.kt, StreamActivity.kt, StreamOptions.kt. | PARTIALLY FIXED. Framing, exact reads, validation, timeout/EOF/replacement cleanup and parser tests added. | Unit tests pass, but malformed packet and process-death behavior were not comprehensively tested on the physical device.

M3-05 · MEDIUM Legacy Windows descriptor/error deserializer lacked byte bounds. Evidence: windows/src/net/server.cpp; serializer.cpp; connection.cpp. | PARTIALLY FIXED. Length-framed, bounded parsing; Windows activation serialization aligned with Android; controller narrowed to loopback/ADB. | Windows was not built or runtime-tested. It is not the active Linux/Android Wi-Fi path.

M3-06 · MEDIUM Linux wildcard listener and heuristic first-interface QR address. Evidence: linux/src/network.cpp; linux/src/main.cpp. | PARTIALLY FIXED. Route-aware IPv4 selection and binding to selected IPv4 address; Windows binds loopback. | IPv6 unsupported. VPN/default-route preference and multiple-active-interface cases untested; no explicit interface selector.

M3-07 · MEDIUM RTSP unauthenticated access and uncertain bind scope. Evidence: Android rtsp/Streamer.kt; Linux security_validation.cpp and main.cpp. | PARTIALLY FIXED. Random per-session path and Basic credentials; Linux validates destination/port/path and derives credential from authenticated session. | RTSP library does not gate every method; wildcard IPv4 bind and lack of TLS remain. Credentials/media can be observed on LAN. Independent direct-media access remains unverified.

M3-08 · LOW Android socket EOF, connect timeout and cleanup gaps. Evidence: Android networking/ConnectionManager.kt; connection/TCPConnection.kt; StreamActivity.kt. | PARTIALLY FIXED. Timeouts, exact frame reads, generation checks, connection replacement and idempotent cleanup. | Full Android disconnect/reconnect/process-death matrix has not been completed.

M3-09 · LOW URLs/endpoints and errors could disclose sensitive metadata. Evidence: linux/src/main.cpp; Android rtsp/Streamer.kt; windows/src/net/*. | PARTIALLY FIXED. Application logs redact/remove sensitive URL, credentials, peer-controlled error text; Android RTSP library logging disabled. | Runtime log review was incomplete. Third-party logging and future diagnostics must remain credential-aware.

M3-10 · MEDIUM Windows activation fields disagreed with Android widths/order. Evidence: windows/src/net/serializer.cpp; Android rtsp/StreamOptions.kt. | PARTIALLY FIXED. Canonical field widths/order, booleans, focus, flash, filter encoding and bitrate scale aligned in source. | Cross-language golden runtime verification and Windows compilation remain.

M3-11 · MEDIUM FFmpeg inherited listener/control descriptors; weak child cleanup. Evidence: linux/src/ffmpeg_runner.cpp; linux/src/rtsp_probe.cpp; linux/src/main.cpp. | PARTIALLY FIXED. CLOEXEC sockets, close-range child cleanup, bounded SIGTERM/escalation/reaping for probe and FFmpeg. | These changes were not revalidated via /proc after remediation or by parent-exit test.

7. Remediation footprint and compatibility

The following is the broad set of implementation areas touched by the recorded remediation and integration pass.

This list is a handoff map, not a claim that each file was edited during PDF generation.

Area | Files / components | Compatibility note

Linux control/security | linux/src/main.cpp, network.cpp, qr.cpp, device_descriptor.cpp, stream_options.cpp, new_control_protocol.cpp/.hpp, new security_validation.cpp/.hpp, CMakeLists.txt, linux/tests/ | Wi-Fi control requires new OCAM1 QR format and protocol v1. Linux now requires OpenSSL Crypto.

Linux process/media | linux/src/rtsp_probe.cpp, ffmpeg_runner.cpp, V4L2/main lifecycle paths | RTSP URL policy and protocol allowlist are intentionally strict; FFmpeg diagnostic visibility remains the next improvement.

Android app/network | MainActivity.kt, QRScanner.kt, ConnectionManager.kt, DeviceDescriptor.kt, Connection.kt, TCPConnection.kt, new ControlProtocol.kt, StreamActivity.kt, StreamOptions.kt, Streamer.kt, pairing UI/resources, protocol tests | Android must be updated together with Linux; old endpoint-only QR is rejected. No new Android dependency was added.

Legacy Windows | windows/src/net/connection.*, server.*, serializer.*, devicedescriptor.h, RTSP manager, device UI, windows/README.md | Windows narrowed to loopback/ADB. Authenticated Wi-Fi protocol was not ported; Windows build remains outstanding.

Audit/worktree | M3_SECURITY_AUDIT.md contains historical findings plus remediation and real-device results. | The existing root README.md staged modification predates/was preserved through audit work; no commit was made.

Protocol migration: Linux/Android Wi-Fi uses OCAM1|IPv4|6969|<64-hex-secret> plus framed protocol v1. Endpoint-only QR and old clients are incompatible. Windows supports only its legacy ADB path until the authenticated Wi-Fi protocol is ported. Pairing secrets are one-use/short-lived; request a fresh QR for each new pairing.

8. Tests and build evidence

Check | Recorded result | Interpretation

Linux configure/build | PASS - cmake -S linux -B linux/build; cmake --build linux/build -j2 | Linux source compiles in the recorded remediation run.

Linux CTest | PASS, 2/2 - opencam_security_tests, opencam_shell_injection_regression | Authentication/frame parsing/validation and harmless metacharacter regression covered. Does not prove all runtime paths.

Android unit tests + debug APK | PASS - ./gradlew testDebugUnitTest assembleDebug | Protocol/token/full-duplex/media-credential tests compile and pass; APK was installed/launched on the physical phone.

Windows build | NOT RUN | Visual Studio/MSBuild unavailable in the Linux environment.

Physical Wi-Fi/QR control path | PASS | USB remained physically connected; explicit Wi-Fi selected; fresh OCAM1 QR and mutual authentication succeeded.

Android camera + RTSP | PASS through RTSP readiness | Android reported stream started; Linux probe returned ready; bounded manual ffmpeg reported H.264 640x480 and AAC. No footage saved.

FFmpeg -> V4L2 frames | FAIL / unresolved | FFmpeg process exited 8; stderr discarded; no frames verified.

USB/ADB Linux path, clean full-pipeline reconnect | NOT VERIFIED | Linux ADB port mapping/support is incomplete; do not infer support from the UI selector.

9. Current physical-device integration trace

-
- Android Wi-Fi mode selected with USB attached - confirmed; Linux accepted the Wi-Fi peer.
 - QR parser extracted the fresh OCAM1 endpoint/secret; mutual control authentication - confirmed.
 - Authenticated device descriptor reached Linux - confirmed.
 - Linux sent activation and Android camera/stream startup succeeded - confirmed.
 - Android RTSP became reachable to the authenticated Linux media client; readiness check and independent bounded ffmpeg succeeded - confirmed.
 - Linux started FFmpeg with RTSP/TCP, protocol whitelist, YUV420P conversion, 640x480 V4L2 output - process launch confirmed.
 - FFmpeg exited with numeric status 8. Because child stdout and stderr are redirected to /dev/null, the decisive diagnostic was discarded. After the child exited, querying /dev/video2 returned VIDIOC_G_FMT: Invalid argument and VIDIOC_G_PARM: Invalid argument.

The last two observations do not establish whether FFmpeg failed to open the device, negotiated an unsupported output format, encountered a driver/caps condition, or hit another output/runtime error. A V4L2 query after no writer is active is not a valid substitute for observing the negotiated format while FFmpeg is open.

10. Manual troubleshooting plan for the unresolved FFmpeg exit

10.1 Preserve security and get a clean baseline

- Keep Android set to Wi-Fi; keep the phone and Linux host on the same Wi-Fi/LAN. The phone may also remain connected by USB; USB must not override Wi-Fi.
- Build both sides from the same repository revision. Never use an old endpoint-only QR or share the one-time QR/session credential.
- Start Linux OpenCam in a visible terminal after rebuilding. Use the fresh QR. Keep all diagnostic material local and redact any URL userinfo/password before sharing it.
- Do not run unbounded FFmpeg loops or repeated stress tests. Stop one session normally before testing device

output independently.

```
cd /home/V3NOM/Projects/OpenCam
cmake -S linux -B linux/build
cmake --build linux/build -j2
ctest --test-dir linux/build --output-on-failure
(cd android && ./gradlew testDebugUnitTest assembleDebug)
adb install -r android/app/build/outputs/apk/debug/app-debug.apk
```

The last adb command is optional for Wi-Fi and requires an authorized device. It is not evidence that Linux USB/ADB mode works.

10.2 Check the V4L2 target before another stream attempt

```
v4l2-ctl --list-devices
for p in /sys/class/video4linux/video*/name; do printf '%s: ' "$p"; cat "$p"; done
ls -l /dev/video2
v4l2-ctl --device=/dev/video2 --all
v4l2-ctl --device=/dev/video2 --list-formats-out-ext
```

Confirm the node named OpenCam is actually /dev/video2, that it is a writable V4L2 loopback output, and that its accepted format/caps include the chosen dimensions and pixel format. If the device node or format is wrong, fix the local v4l2loopback module/device setup first. Do not assume node number 2 is stable; the application discovers by the OpenCam card label.

10.3 Capture the actual FFmpeg error safely

Current linux/src/ffmpeg_runner.cpp::startFfmpeg deliberately duplicates /dev/null onto FFmpeg stdout and stderr. The correct next engineering change is a small, bounded diagnostic channel: pipe FFmpeg stderr to the parent or write to a local owner-only file, keep only the final limited number of lines, and redact URL userinfo/control characters before logging. Do not log the full RTSP URL or session-derived password.

For a one-off manual diagnosis, a developer can temporarily redirect only FFmpeg stderr to a file created with owner-only permissions, rebuild, run one test, inspect locally, then delete that log. FFmpeg errors may contain authenticated URLs; never paste the raw file into tickets/chat or leave it in a shared directory. Revert the temporary diagnostic change afterward. Prefer a pipe with a fixed-size ring buffer for a durable fix, to avoid disk growth and secret retention.

Record just: FFmpeg version, exit code or signal, error category/message after redaction, V4L2 node identity, negotiated dimensions/pixel format, whether the input connected, and whether an output frame was written.

10.4 Isolate the V4L2 sink from phone/network

Once no OpenCam/FFmpeg writer is active and the target device has been confirmed, a bounded local synthetic-source write can test the loopback sink independently of RTSP. It is a diagnostic, not a permanent fix; it may temporarily occupy the virtual camera, so close consumers and stop the test after a few seconds.

```
ffmpeg -hide_banner -f lavfi -i testsrc=size=640x480:rate=30 \
-t 3 -vf format=yuv420p -an -f v4l2 -pix_fmt yuv420p /dev/video2
```

If this fails, diagnose the V4L2 module, device node, permissions, exclusive-caps behavior, and output format before changing RTSP. If it succeeds, the sink can accept a basic test pattern, so compare the live FFmpeg input parameters and current V4L2 caps. Do not run the test while a real stream is active.

10.5 Re-test the actual stream, then verify frames rather than just sockets

- Launch the rebuilt Linux server from a terminal; select Wi-Fi on Android; scan the fresh QR and connect.
- Confirm sanitized log sequence: authentication accepted -> descriptor accepted -> activation sent -> RTSP ready -> FFmpeg started.
- Capture a redacted FFmpeg error if it exits. If it remains running, verify /dev/video* name/format while the process is alive, and use a local consumer or supported v4l2-ctl capture check to confirm frames arrive.
- Stop normally and confirm FFmpeg exits and Linux returns to its listener state. Then perform one reconnect only after output has been verified.

Do not call "RTSP ready" a webcam success. It means the media input probe succeeded. The acceptance criterion is a live FFmpeg writer plus observable frames at the OpenCam V4L2 node and a consumer receiving them.

11. Proposed changes / remaining security work

Priority | Proposal | Why / acceptance check

P0 · debug/operability | Add bounded FFmpeg stderr capture with credential redaction; log distinct startup, input-open, output-open, and frame-write failure states rather than only a numeric exit. | Identifies the current status-8 failure without exposing secrets. Test malformed RTSP locally, invalid V4L2 output, normal stream, and confirm no credentials in logs.

P0 · pipeline validation | Check V4L2 node capability, write access, format and dimensions before/after FFmpeg startup; return an actionable error if the node is not a writable capture-loopback output. | Prevents false-positive "stream ready" messaging. Require known-good 3-second test pattern and phone stream to produce observed frames.

P1 · control confidentiality | Add a standard TLS transport or an authenticated encrypted tunnel with pinned server identity; keep current pairing proof as authorization/bootstrap, not as a substitute for encryption. | Prevents LAN observers from reading control metadata/pairing material and reduces active MITM risk. Test wrong pin, expired secret, replay, and on-path integrity/confidentiality.

P1 · RTSP media boundary | Replace or wrap the current RTSP library so every method is authorized and the listener can bind narrowly; alternatively tunnel media over the authenticated session. Keep per-session random path/password and invalidate on disconnect. | Current library has incomplete method authorization, wildcard IPv4 bind, and no TLS. Test OPTIONS/DESCRIBE/SETUP/PLAY with and without credentials from authorized and unauthorized local clients.

P1 · USB/ADB support clarity | Either mark USB mode as unsupported for Linux until integrated, or implement host-side ADB reverse for control and forward for RTSP, bind/route safely, and use the same authenticated framing. | USB selector currently proves explicit choice only, not Linux connectivity. Acceptance: test USB selected with Wi-Fi unavailable, verify mutual auth and camera frames, then verify Wi-Fi selected with USB plugged in still uses QR.

P2 · protocol availability | Reduce serial client starvation further with a bounded accept/handshake worker or explicit busy rejection; retain strict per-peer deadlines and a small concurrent cap. | Current finite timeouts are much better than indefinite occupancy, but a peer can still delay a legitimate one. Test slow/incomplete clients and bounded simultaneous connections.

P2 · network scope | Offer an explicit intended-interface selection or bind policy; add deliberate IPv6 support only with equivalent auth/testing, otherwise document IPv4-only clearly. | Route-aware address choice can still follow a VPN/default route. Test Wi-Fi + VPN + virtual interfaces without exposing the listener beyond the chosen interface.

P2 · cross-platform quality | Add Windows CI/build and golden serialization vectors; retain ADB-only loopback policy until authenticated Wi-Fi is ported. | Source corrections alone do not establish Windows portability. Test canonical field bytes on both Windows and Android.

P3 · operational assurance | Add repeatable local integration checks for stale QR/token, disconnect, reconnect, Android death, FFmpeg failure, inherited FDs, and stale media credentials. | Builds and unit tests do not cover process lifecycle. Observe socket/FD counts and verify cleanup on every terminal path.

12. Manual reproducibility commands

From the repository root, the recorded build/test sequence is:

```
cmake -S linux -B linux/build
cmake --build linux/build -j2
ctest --test-dir linux/build --output-on-failure
(cd android && ./gradlew testDebugUnitTest assembleDebug)
adb install -r android/app/build/outputs/apk/debug/app-debug.apk
```

For a Wi-Fi hardware run: start `./linux/build/opencam` in a terminal; ensure host and phone share a LAN; on Android explicitly select Wi-Fi; scan the current one-use OCAM1 QR; connect; inspect sanitized Linux/Android logs; verify a process and then actual frames at the OpenCam V4L2 node. Do not publish QR contents, session URLs, or raw process command lines.

Read-only local state checks, when no session is being disrupted:

```
ip -br addr
ip route
ss -ltnp '( sport = :6969 )'
pgrep -x opencam
pgrep -x ffmpeg
pgrep -x ffprobe
v4l2-ctl --list-devices
```

Use `pgrep -x` rather than dumping full process command lines because the RTSP input argument contains per-session credentials.

13. Final status and decision

Question | Evidence-based answer

Is the original QR/Wi-Fi connection failure fixed? | Yes, for Wi-Fi: real phone completed OCAM1 mutual authentication with USB still attached and Wi-Fi explicitly selected.

Did descriptor/activation complete? | Yes: authenticated descriptor accepted and activation sent; Android started the stream.

Does Linux reach the Android RTSP source? | Yes: readiness check and manual ffprobe succeeded; video stream metadata was identified.

Does the virtual camera receive frames? | Not established: FFmpeg exited 8 before frame delivery was confirmed; V4L2 query afterward was invalid.

Is USB/ADB mode working with Linux? | Not verified / implementation incomplete: do not claim it works end-to-end.

Is OpenCam secure for hostile networks? | No such claim is justified. Authentication and validation improved, but control/media encryption and complete RTSP authorization remain limitations.

Practical next step: diagnose V4L2 and expose bounded, redacted FFmpeg stderr. Do not revisit pairing/authentication unless a new failure points there; those stages already succeeded in the recorded physical-device run.

Detailed original findings and historical evidence: M3_SECURITY_AUDIT.md. This PDF is a generated handoff and does not replace that audit. No source files were changed to create this document.